normalization

idea is to maximize precision when using a fixed number of digits, by ensuring no leading zeros

- e.g. suppose b=10 and are using four digits to the right of the decimal point ... rather than storing 13 1/3 as 0.0133×10^3, would be better to store as 1.3333×10^1
- use *shifting* to get normalized representation:
  decrease exponent by 1 for each shift of the base point to the right
  increase exponent by 1 for each shift of the base point to the left

issues in floating point number representation

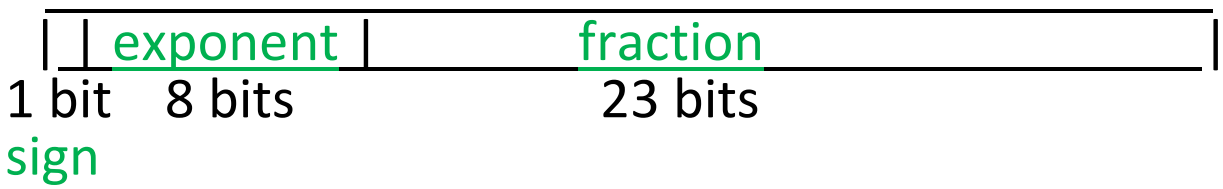
- what needs to be stored?
- choice of the base b
- number of bits used for the digits, number of bits used for exponent
- representations used for the (signed) exponent and digits

need *standards* to ensure software portability …

IEEE 754 binary floating point standard

basic formats include single precision (one word), double precision (two words) and quad precision (four words)

single precision format:



double precision has 11 bit exponent field, 52 bit fraction field

quad precision has 15 bit exponent field, 112 bit fraction field

fraction field includes only the digits to the right of the binary point

- a normalized binary floating point number will always have a leading digit of 1, so don't need to store that digit!

exponent represented using biased notation

- idea is to add a bias to actual exponent value, equal to absolute value of smallest (negative) exponent one wants to be able to represent
- after adding bias, will be non-negative, so can represent as an unsigned binary number
- with n-bit exponent, if use bias of $2^{n-1} - 1$ (e.g., for single precision, bias of 127), can represent values between $-(2^{n-1} - 1)$ and 2^{n-1}

to find the representation of an exponent: add the bias to the exponent

to find the exponent being represented: subtract the bias from the contents of the exponent field

examples of representing numbers using the IEEE 754 single precision format

-748.25_{10}

separately convert the integer and fractional parts to binary:

$-748.25_{10} = -1011101100.01_2$

write in normalized scientific notation:

$-1011101100.01_2 = -1.01110110001_2 \times 2^9$

add 127 (the bias for the single precision format) to the exponent, and convert to binary:

$127 + 9 = 136_{10} = 10001000_2$

now, just fill in the fields, giving:

1	10001000	01110110001000000000000

0.3_{10}

to convert to binary, repeatedly multiply by 2, taking the integer part of each result as a digit in the binary expansion, and continuing with the fractional part:

0.3 x 2 = 0.6, 0.6 x 2 = 1.2, 0.2 x 2 = 0.4, 0.4 x 2 = 0.8, 0.8 x 2 = 1.6, 0.6 x 2 = ...

notice pattern starting to repeat, so:

$0.3_{10} = 0.0\overline{1001}_2$

write in normalized scientific notation:

$0.0\overline{1001}_2 = 1.\overline{0011}_2 \times 2^{-2}$

add 127 to the exponent, and convert to binary:

$127 + -2 = 125_{10} = 01111101_2$

finally, fill in the fields:

| 0 | 01111101 | 00110011001100110011001 |

here we've assumed that truncation is used; if rounding used instead, which it typically is, would get ...010 at end instead of ...001

special cases in the IEEE 754 floating point formats (below description assumes single precision)

exponent field is all zeros

- the exponent being represented is -126 (for single precision), the same exponent as when the exponent field contains 0...01
- have an implicit leading 0 (rather than 1); such numbers called "denorms"
- can now represent zero by 000...0
- smallest positive number we can represent (for single precision) is now $0.000 \ldots 1 \times 2^{-126}$ rather than $1.000 \ldots 0 \times 2^{-127}$

exponent field is all ones

- fraction part is all zeros: represents (positive or negative) infinity
- fraction part is non-zero: represents NaN; NaN used when result of an operation is completely unknown (e.g. x/y when both x and y have "underflowed")